

Numerical Methods From Scratch: Engineering Mathematics Implemented in Modern C++

Gemini CLI Engineering Team

June 2, 2026

Abstract

This report details the design and implementation of a professional scientific computing library built from first principles using the C++20 standard. The project, *Numerical Methods From Scratch*, emphasizes mathematical rigor, numerical stability, and computational efficiency. We cover a broad spectrum of numerical analysis, including root-finding, linear algebra, interpolation, calculus, and differential equations. The implementation leverages modern C++ features, RAII, and template metaprogramming to provide a robust framework for engineering and mathematical computations.

Contents

1 Introduction

The field of numerical analysis is foundational to modern engineering and scientific discovery. While high-level environments like MATLAB or Python’s NumPy provide powerful abstraction, there is significant value in implementing core algorithms from scratch to understand their limitations, stability characteristics, and performance trade-offs.

This project aims to bridge the gap between abstract mathematical theory and concrete software implementation. By using C++20, we take advantage of advanced features such as concepts, ranges, and improved template support to create a type-safe and performant library. The primary dependencies include **Eigen** for matrix operations, **matplotlib** for visualization, and **GoogleTest** for rigorous verification.

2 Root Finding Methods

Root-finding involves finding x such that $f(x) = 0$. We implemented four fundamental methods.

2.1 Bisection Method

The Bisection Method relies on the Intermediate Value Theorem. For a continuous function f , if $f(a)f(b) < 0$, a root exists in $[a, b]$.

$$c = \frac{a + b}{2} \tag{1}$$

While slow (linear convergence), it is unconditionally stable.

2.2 Newton-Raphson Method

Newton’s method uses the first-order Taylor expansion:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \tag{2}$$

It offers quadratic convergence ($e_{n+1} \approx Ce_n^2$) but requires the existence and calculation of the derivative.

2.3 Secant Method

The Secant method approximates the derivative using two points:

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})} \tag{3}$$

It avoids analytical derivatives while maintaining superlinear convergence ($\phi \approx 1.618$).

3 Linear Algebra

Solving $Ax = b$ is the workhorse of scientific computing.

3.1 LU Decomposition

We implemented Doolittle’s algorithm to factor $A = LU$, where L is unit lower triangular and U is upper triangular.

$$A = \begin{pmatrix} 1 & 0 \\ l_{21} & 1 \end{pmatrix} \begin{pmatrix} u_{11} & u_{12} \\ 0 & u_{22} \end{pmatrix} \tag{4}$$

This allows solving multiple systems with different b in $O(n^2)$ time after an initial $O(n^3)$ decomposition.

3.2 Iterative Solvers

For large sparse systems, we implemented Jacobi and Gauss-Seidel methods. Gauss-Seidel uses the iteration:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right) \quad (5)$$

Convergence is guaranteed for strictly diagonally dominant matrices.

4 Interpolation

Interpolation fits a function to discrete data points.

4.1 Cubic Spline Interpolation

Cubic splines provide a piecewise cubic polynomial $S(x)$ that is C^2 continuous.

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 \quad (6)$$

Unlike high-degree global polynomials, splines avoid Runge's phenomenon and provide smooth approximations.

5 Numerical Differentiation

We utilize finite difference quotients to approximate derivatives.

- **Central Difference:** $f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} + O(h^2)$

The central difference is preferred for its second-order accuracy.

6 Numerical Integration

Quadrature rules approximate $\int_a^b f(x)dx$.

6.1 Gaussian Quadrature

Gaussian quadrature selects optimal nodes x_i and weights w_i :

$$\int_{-1}^1 f(x)dx \approx \sum_{i=1}^n w_i f(x_i) \quad (7)$$

An n -point rule is exact for polynomials of degree $2n - 1$.

7 Ordinary Differential Equations

We solve Initial Value Problems (IVPs) of the form $y' = f(t, y)$.

7.1 Runge-Kutta 4 (RK4)

RK4 is a high-accuracy fourth-order method:

$$k_1 = f(t_n, y_n) \quad (8)$$

$$k_2 = f(t_n + h/2, y_n + hk_1/2) \quad (9)$$

$$k_3 = f(t_n + h/2, y_n + hk_2/2) \quad (10)$$

$$k_4 = f(t_n + h, y_n + hk_3) \quad (11)$$

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (12)$$

8 Error Analysis

Numerical errors are classified into:

- **Truncation Error:** From approximating an infinite process with a finite one.
- **Round-off Error:** From finite floating-point precision.

We utilize *Result* < *T* > structures to track the convergence history and residual errors of every computation.

9 Results and Visualizations

We generated several plots to validate our implementations. (Note: Plots reference the `assets/images/` directory).

9.1 Convergence of Root Finding

The comparison between Bisection and Newton-Raphson highlights the quadratic nature of Newton's method. While Bisection requires ~ 40 iterations for 10^{-12} precision, Newton's method converges in fewer than 10.

9.2 ODE Solver Accuracy

Plotting the solution of $y' = y$ demonstrates that RK4 remains highly accurate even with relatively large step sizes h , whereas the Euler method accumulates significant local truncation error.

10 Conclusion

The *Numerical Methods From Scratch* library demonstrates that modern C++20 is an excellent choice for scientific computing. By combining mathematical rigor with robust engineering practices, we have created a library that is both educationally valuable and computationally sound. Future work could include the implementation of implicit ODE solvers for stiff systems and multi-dimensional root-finding.

References

- [1] Burden, R. L., & Faires, J. D. (2010). *Numerical Analysis*. Cengage Learning.
- [2] Knuth, D. E. (1997). *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley.
- [3] ISO/IEC. (2020). *ISO/IEC 14882:2020 Programming languages — C++*.